key is *not* a symbol. One other thing to watch out for—if the *value* is a symbol, you'll need to have at least one space between the colons or else you'll get a syntax error:

```
inst_section = {
  cello:    :string,
  clarinet: :woodwind,
  drum:     :percussion,
  oboe:     :woodwind,
  trumpet:  :brass,
  violin:   :string
}
```

Hashes are indexed using the same square bracket notation as arrays:

```
inst_section[:oboe]     #=> :woodwind
inst_section[:cello]    #=> :string
inst_section[:bassoon]  #=> nil
```

As the preceding example shows, a hash returns `nil` when indexed by a key it doesn't contain. Normally this is convenient because `nil` means false when used in conditional expressions.

You can pass hashes as parameters on method calls. Ruby allows you to omit the braces but only if the hash is the last parameter of the call. Rails makes extensive use of this feature. The following code fragment shows a two-element hash being passed to the `redirect_to` method. Note that this is the same syntax that Ruby uses for keyword arguments:

```
redirect_to action: 'show', id: product.id
```

One more data type is worth mentioning: the regular expression.

Regular Expressions

A regular expression lets you specify a *pattern* of characters to be matched in a string. In Ruby, you typically create a regular expression by writing `/pattern/` or `%r{pattern}`.

For example, we can use the regular expression `/Perl|Python/` to write a pattern